YouthPath 시스템 동작 프로세스

YouthPath 시스템 동작 프로세스

> 사용자 자연어 질의 1번으로 청년정책·채용공고·자소서 가이드·마감일을 통합 응답하는 Multi-Agent 시스템이 실제로 어떻게 동작하는지 처음부터 끝까지의 흐름.

---

0. 전체 흐름 한 장

```
[사용자 입력]
   ↓
[FastAPI 백엔드] ─── DynamoDB (프로필·대화기록)
   ↓
[LangGraph Router]
   ├─ ① 분류 LLM (LUXIA)
   ├─ ② Policy Agent    ┐
   ├─ ② Job Agent       ├─ 병렬 실행 (LLM 안 씀)
   ├─ ② Resume Agent    │
   ├─ ② Calendar Agent  ┘
   ├─ ③ 포맷터 (JSON → 자연어 텍스트)
   └─ ④ 통합 LLM (LUXIA)
   ↓
[Streamlit Frontend] — 답변 텍스트 + 5탭 카드
```

LLM은 단 2번만 호출: 맨 앞 분류 1번 + 맨 뒤 통합 1번. 그 사이 4개 Agent는 전부 코드(API·룰·점수·벡터검색).

---

1. 예시 시나리오로 따라가기

사용자 질의:

> "서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘"

사용자 프로필 (사이드바에서 1번만 입력):

```
나이 27, 서울, 중위소득 60% 이하, Python/SQL, 데이터 분석가, 관심 기업: 네이버
```

---

2. STEP 1 — 입력이 백엔드로 전달

Streamlit이 FastAPI에 HTTP POST를 보냄:

```
POST /ask
{
  "user_id": "demo_001",
  "query": "서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘",
  "profile": {
    "age": 27,
    "region": "서울",
    "skills": ["Python", "SQL"],
    "target_role": "데이터 분석가",
    "target_company": "네이버"
  }
}
```

FastAPI가 받자마자 다음을 수행:

1. DynamoDB `UserProfiles` 에서 기존 프로필 조회 (있으면 합침)
2. DynamoDB `Conversations` 에 "user 메시지" 저장
3. LangGraph Router 그래프 호출

---

3. STEP 2 — 분류 LLM (LLM 호출 1번째)

LUXIA에 짧은 분류용 프롬프트 1개를 던짐:

```
사용자 질의를 분석하여 호출할 에이전트를 모두 선택하세요.

에이전트:
- policy: 청년 정책, 지원금, 복지
- job: 채용공고, 직무, 회사 추천
- resume: 자소서 가이드
- calendar: 마감일, 일정

JSON 형식: {"agents": [...], "reasoning": "..."}

사용자 질의: "서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘"
```

LUXIA 응답:

```
{ "agents": ["policy", "job"], "reasoning": "정책+채용 복합 질의" }
```

→ "policy, job 두 개를 부르자"가 결정됨. (resume, calendar는 이번엔 skip)

---

4. STEP 3 — 각 Agent 병렬 실행 (LLM 안 씀)

4개 Agent는 **LLM을 절대 안 쓰고** 각자 자기 방식으로 데이터를 모음. LangGraph가 `asyncio.gather` 로 동시 실행.

4-A. Policy Agent

```
1. Chroma `policies` 벡터 컬렉션에서 의미 검색
   query="월세 지원", filter={"region": "서울"}, k=10
   → 후보 정책 청크 10개

2. 정책 단위로 그룹화
   같은 정책의 청크들을 묶어서 후보 정책 객체 생성

3. 자격 매칭 룰 (LLM 없이 순수 코드 수식)
   각 후보 정책마다:
     score = 0
     if 정책.min_age <= 27 <= 정책.max_age:  score += 0.4
     if "서울" in 정책.region:                score += 0.3
     if 사용자.소득 ∈ 정책.소득기준:           score += 0.3

4. score >= 0.7 인 정책만 남김
   matched_criteria / unmatched_criteria 필드 채움
```

출력 items 예시:

```
[
  {
    "title": "청년월세지원",
    "deadline": "2026-05-31",
    "matched_criteria": ["나이 27세 ✓", "서울 ✓", "중위 60% ✓"],
    "unmatched_criteria": [],
    "link": "https://..."
  },
  { "title": "주거안정월세대출", "...": "..." }
]
```

4-B. Job Agent

```
1. Worknet + Saramin API를 asyncio.gather로 동시 호출
   query: "데이터 분석가", region: "서울", limit: 10

2. 두 API 응답을 UnifiedJob 형식으로 통일
   필드명·날짜 형식·카테고리 차이를 어댑터가 흡수

3. 중복 제거
   같은 회사 + 같은 직무는 중복으로 보고 하나만 남김

4. 적합도 점수 계산 (LLM 없이 수식)
   score = 0.4 * 스킬_매칭률
         + 0.3 * 경력_매칭
         + 0.2 * 지역_매칭
         + 0.1 * 마감_긴급도

5. 점수 내림차순 정렬 → 상위 5개
```

API 실패 대응: - 5초 timeout + 3회 지수 백오프 재시도 - Circuit Breaker: 연속 실패 시 일시 차단 - 일부 API만 실패하면 다른 결과로 응답 + `metadata` 에 알림 표시

출력 items 예시:

```json
[
  {
    "title": "데이터분석 신입",
    "company": "○○회사",
    "location": "서울 강남",
    "deadline": "2026-05-20",
    "fit_score": 0.87,
    "url": "https://..."
  }
]
```

4-C. Resume Agent

이번 질의에선 분류 LLM이 호출 안 했지만, 호출됐다면 다음처럼 동작:

```
1. 화이트리스트 체크
   profile.target_company("네이버")가 사전 인덱싱된 100개 화이트리스트에 있는지 확인
   → 없으면 즉시 fallback: "이 기업은 인덱싱되지 않았습니다" 메시지로 종료

2. Chroma `companies` 벡터 컬렉션 검색
   query="사업의 내용 R&D 인재상", filter={"company_name": "네이버"}, k=15
   → DART 사업보고서에서 추출된 청크 15개

3. 청크 합치기
   15개 청크를 하나의 큰 텍스트로 이어붙임
   (사업 내용 + 연구개발 + 인재상 섹션)

4. KeyBERT로 키워드 추출 (LLM 아님, 임베딩 기반)
   합친 텍스트에서 회사 핵심 키워드 15개 추출
   예: ["검색", "AI", "추천 시스템", "광고", "글로벌", "클라우드"]

5. 매칭 알고리즘 (코사인 유사도)
   사용자 skills(Python, SQL)와 회사 키워드 간 임베딩 유사도 계산
   유사도 >= 0.6 인 페어만 매칭으로 인정
   예: Python ↔ "AI" (0.82), SQL ↔ "추천 시스템" (0.71)

6. 부족 영역(gaps) 도출
   매칭 안 된 회사 키워드 = 사용자가 자소서에서 보강해야 할 부분
   예: ["대규모 트래픽 경험", "광고 도메인"]

7. Story Angle 룰 매핑
   키워드 패턴 → 사전 정의된 5개 angle 중 매핑
   예: "AI·R&D" 키워드 다수 → "기술적 깊이" angle 추천
```

출력 items (1개 회사당 1개):

```json
{
  "company": "네이버",
  "emphasize_keywords": ["검색", "AI", "추천 시스템"],
  "matching_points": [
    {"user_skill": "Python", "company_keyword": "AI", "fit_score": 0.82},
    {"user_skill": "SQL", "company_keyword": "추천 시스템", "fit_score": 0.71}
  ],
  "evidence_gaps": ["대규모 트래픽 경험", "광고 도메인"],
  "story_angles": ["기술적 깊이"]
}
```

핵심: 자소서를 **써주지 않음**. "어떤 키워드를 강조하고, 뭘 보강하고, 어떤 각도로 풀어라"는 **가이드라인만** 제공.

4-D. Calendar Agent

다른 Agent와 다르게 **"수집"이 아니라 "정리"하는 Agent**.

```
1. DynamoDB `UserEvents` 테이블 조회
   key: user_id="demo_001"
   → 사용자가 과거에 저장해둔 이벤트 목록 가져옴

2. 다른 Agent 결과에서 마감일 자동 흡수
   - Policy 결과의 각 item.deadline
   - Job 결과의 각 item.deadline
   → 정규식으로 날짜 추출 (서비스 함수 extract_deadline())
   → 새 이벤트로 DynamoDB UserEvents에 저장

3. 마감일 필터링
   compute_days_remaining(deadline) < 0 인 건 제외 (이미 지난 일정)

4. D-day 계산
   각 이벤트에 days_remaining 필드 추가

5. 정렬
   D-day 짧은 순으로 sort
```

출력 items 예시:

```json
[
  {
    "title": "○○회사 데이터분석 신입 마감",
    "deadline": "2026-05-20",
    "days_remaining": 8,
    "link": "https://..."
  },
  {
    "title": "청년월세지원 마감",
    "deadline": "2026-05-31",
    "days_remaining": 19,
    "link": "https://..."
  }
]
```

4개 Agent 동작 방식 비교

Agent	데이터 출처	핵심 알고리즘	LLM
Policy	Chroma `policies` (Static RAG)	자격 매칭 룰 (나이/지역/소득 점수)	❌
Job	Worknet/Saramin API (Dynamic, 실시간)	적합도 점수 (스킬·경력·지역·마감 가중합)	❌
Resume	Chroma `companies` (Static RAG, DART)	KeyBERT 키워드 추출 + 코사인 유사도 매칭	❌

Agent	데이터 출처	핵심 알고리즘	LLM
Calendar	DynamoDB `UserEvents` + 다른 Agent 결과	정규식·날짜 계산·D-day 정렬	❌

---

5. STEP 4 — 각 Agent의 응답 JSON 형식 (도메인마다 다름)

모든 Agent는 **같은 외피**로 응답하되, `items` 안의 키는 도메인마다 다름:

공통 외피 (BaseAgent가 강제)

```
{
  "agent_name": "policy",       # | "job" | "resume" | "calendar"
  "items": [ ... ],             # ← 이 안의 키는 도메인마다 다름
  "sources": [ ... ],
  "metadata": { "latency_ms": 320 },
  "error": null
}
```

Policy items 키

```
{ "title", "deadline", "matched_criteria", "unmatched_criteria", "link" }
```

Job items 키

```
{ "title", "company", "location", "deadline", "fit_score", "url" }
```

Resume items 키

```
{ "company", "emphasize_keywords", "matching_points",
  "evidence_gaps", "story_angles" }
```

Calendar items 키

```
{ "title", "deadline", "days_remaining", "link" }
```

왜 통일 안 하나?

정책엔 "자격 충족 조건"이 핵심이지만 채용엔 "적합도 점수", 자소서엔 "키워드 매칭", 일정엔 "D-day"가 핵심임. 억지로 키를 통일하면 의미가 사라져서 일부러 도메인별로 둠.

대신 **외피(`agent_name`, `sources`, `error`)는 통일**해서 Router가 어떤 Agent든 똑같은 방식으로 다룰 수 있게 함.

---

6. STEP 5 — 통합 LLM에 넣기 전, JSON을 자연어로 변환 (포맷터)

여기가 **"JSON 형식이 다른데 어떻게 합치냐"의 답**.

왜 JSON을 그대로 LLM에 안 넣나

1. 중괄호·따옴표·영어 키 이름이 **토큰 낭비**
2. LLM이 영어 키를 무시하거나 잘못 해석하기 쉬움
3. LLM이 빈 영역을 **망상으로 채울 위험**

해결: Agent별 포맷터 함수

Router에 도메인별 포맷터 함수가 있어서 JSON → 한국어 텍스트로 변환.

Policy 포맷터 결과:

```
1. 청년월세지원
   - 마감: 2026-05-31
   - 자격 충족: 나이 27세, 서울, 중위 60%
   - 부족 요건: 없음
   - 신청: https://...
2. 주거안정월세대출
   - ...
```

Job 포맷터 결과:

```
1. ㅇㅇ회사 - 데이터분석 신입
   - 지역: 서울 강남, 마감: 2026-05-20
   - 적합도: 87%
   - 공고: https://...
```

Resume 포맷터 결과:

```
대상 기업: 네이버
강조 키워드: 검색, AI, 추천 시스템
매칭 포인트:
   - Python ↔ AI (적합도 82%)
   - SQL ↔ 추천 시스템 (적합도 71%)
보강 제안: 대규모 트래픽 경험, 광고 도메인
추천 Story Angle: 기술적 깊이
```

Calendar 포맷터 결과:

```
- ○○회사 데이터분석 신입 마감: D-8 (마감 2026-05-20)
- 청년월세지원 마감: D-19 (마감 2026-05-31)
```

호출 안 된 Agent는 `(해당 결과 없음)` 한 줄로 처리.

---

7. STEP 6 — 통합 LLM (LLM 호출 2번째)

위에서 포맷팅한 텍스트들을 **하나의 큰 프롬프트 슬롯**에 끼워서 LUXIA에 1번 호출.

LUXIA가 받는 최종 프롬프트

```
당신은 청년 사회진입 어시스턴트입니다.
사용자 질문과 아래 전문 에이전트들의 결과를 바탕으로
자연스러운 한국어 답변을 작성하세요.

[사용자 질문]
서울에서 월세 도와주는 정책이랑 IT 신입 공고 알려줘

[사용자 프로필]
27세, 서울, 중위 60%, Python/SQL, 데이터 분석가

[정책 결과]
1. 청년월세지원
   - 마감: 2026-05-31
   - 자격 충족: 나이 27세, 서울, 중위 60%
   - 부족 요건: 없음
   - 신청: https://...
2. 주거안정월세대출
   - ...

[채용 결과]
1. ○○회사 - 데이터분석 신입
   - 지역: 서울 강남, 마감: 2026-05-20
   - 적합도: 87%
   - 공고: https://...

[자소서 결과]
(해당 결과 없음)

[일정 결과]
(해당 결과 없음)

답변 작성 규칙:
1. 친근하지만 전문적인 어조
2. 각 항목 끝에 출처 인용
3. 카드/표가 아닌 자연스러운 문단
4. 사용자가 행동할 구체적 다음 단계 제시
5. "(해당 결과 없음)" 영역은 언급하지 말 것
6. 위에 없는 정보는 절대 만들지 말 것 (할루시네이션 금지)
```

LUXIA 응답

```
서울 거주 27세 기준으로 신청 가능한 월세 지원 2건과
데이터 분석 신입 공고 5건을 정리했어요.

청년월세지원은 자격 요건 모두 충족돼서 가장 유리합니다.
신청 마감이 5월 31일까지라 D-19 안에 신청하셔야 해요.
[출처: 온통청년 API]

채용은 ○○회사 데이터분석 신입이 본인 Python·SQL 기술과
적합도 87%로 가장 잘 맞습니다. 마감이 5월 20일로 임박해요.
[출처: 워크넷·사람인]

다음 단계:
1. 청년월세지원 신청 →
2. ○○회사 지원 →
```

---

8. STEP 7 — 응답 반환 + 화면 렌더링

FastAPI 응답:

```
{
  "answer": "LUXIA가 만든 자연어 답변 ↑",
  "policy": { "items": [...] },
  "job":    { "items": [...] },
  "resume": null,
  "calendar": null
}
```

동시에 DynamoDB `Conversations`에 assistant 메시지 저장 + 발견된 마감일을 `UserEvents`에 자동 저장.

Streamlit 화면: 두 층으로 표시

```
┌──────────────────────────────────────┐
│ 💬 답변 (LUXIA 자연어)                │ ← 위층: 흐름·추천 이유
│                                      │
│ 청년월세지원이 자격 모두 충족해서    │
│ 가장 유리합니다. 마감 D-19...        │
│ 채용은 ○○회사가 적합도 87%로 1위... │
└──────────────────────────────────────┘


┌────────┬────────┬────────┬────────┬────────┐
│ 📋 정책 │ 💼 채용 │ ✍️ 자소서│ 📅 일정 │ 📊 평가 │   ← 아래층: 5개 탭
├────────┴────────┴────────┴────────┴────────┘
│
│   ┌──────────────────────────┐
│   │ 청년월세지원              │  ← 카드 1
│   │ 📅 마감: 2026-05-31       │
│   │ ✓ 충족: 나이, 서울, 중위 60% │
│   │ ✗ 미충족: 없음            │
│   │ [신청하러 가기 →]         │
│   └──────────────────────────┘
│
│   ┌──────────────────────────┐
│   │ 주거안정월세대출          │  ← 카드 2
│   │ ...                       │
│   └──────────────────────────┘
│
```

"raw JSON 카드"의 의미

각 Agent가 만든 `items: list[dict]` (raw JSON)을 **사용자가 보기 좋게 시각 컴포넌트(카드)로 그려주는 것**.

예를 들어 Policy items가 이렇게 들어오면:

```
{
  "title": "청년월세지원",
  "deadline": "2026-05-31",
  "matched_criteria": ["나이 27세 ✓", "서울 ✓", "중위 60% ✓"],
  "unmatched_criteria": [],
  "link": "https://..."
}
```

Streamlit이 JSON을 그대로 보여주지 않고, **렌더링 함수**(`render_policy_card`)가:

- `title` → 큰 글씨 제목
- `deadline` → 📅 아이콘 + 캡션
- `matched_criteria` → 초록색 ✓ 배지
- `unmatched_criteria` → 노란색 ✕ 배지
- `link` → 클릭 가능한 버튼

으로 화면에 그려줌.

왜 자연어 답변이랑 카드 둘 다 보여주나

위층 (LUXIA 답변)	아래층 (카드)
이해를 위한 것	**행동**을 위한 것
"이 정책이 왜 너에게 유리한지" 자연어 설명	마감일·점수·링크 등 **정확한 사실**
우선순위·추천 이유	한눈에 비교 가능한 시각화
흐름·문맥	클릭해서 바로 신청

LUXIA는 **사람처럼 풀어 말해주는 역할**, 카드는 **데이터를 한눈에 보고 다음 행동(클릭)으로 넘어가게 하는 역할**.

도메인별 카드 모양

- **정책 카드**: 자격 충족/미충족 배지 + 마감일 + 신청 버튼
- **채용 카드**: 회사명 + 적합도 % 메트릭 + 위치 + 마감일 + 공고 버튼
- **자소서 카드**: 강조 키워드 + 매칭 페어 + 보강 제안 + Story Angle
- **일정 카드**: D-day 큰 메트릭 + 마감일 + 바로가기 버튼

---

9. 데이터 저장 위치

데이터	어디	왜
정책 본문 임베딩	Chroma `policies`	잘 안 변함 — 미리 인덱싱 (Static RAG)
회사 사업보고서 임베딩	Chroma `companies`	잘 안 변함 — 미리 인덱싱 (Static RAG)
채용공고	저장 안 함	시시각각 변함 — 매번 API 호출 (Dynamic)
사용자 프로필	DynamoDB `UserProfiles`	개인 데이터 영속
대화 기록	DynamoDB `Conversations`	멀티턴 컨텍스트
마감일 일정	DynamoDB `UserEvents`	Calendar Agent가 사용

데이터 처리 3단 분리 원칙: - 잘 안 변하는 건 미리 임베딩해서 검색 → **Static RAG** - 시시각각 변하는 건 캐싱하지 말고 매번 새로 → **Dynamic RAG / API** - 사용자 개인 데이터는 → **DynamoDB**

---

10. 한 줄 요약 + 핵심 트릭

한 줄 요약

```
입력 → 분류 LLM (Agent 고르기)
     → 선택된 Agent들이 코드로 자기 JSON 만듦 (형식은 도메인마다 다름)
     → 포맷터가 JSON을 사람말 텍스트로 변환
     → 통합 LLM이 그 텍스트들을 합쳐 자연어 응답 생성
     → 화면에 답변 텍스트 + raw JSON 카드로 표시
```

핵심 트릭 3개

1. **외피만 통일** — Agent 응답은 같은 wrapper(`agent_name` / `items` / `sources`), 내부 items는 도메인별 자유
2. **JSON을 LLM에 그대로 안 줌** — 포맷터가 도메인별로 자연어로 미리 풀어줌
3. **빈 결과는 `(해당 결과 없음)` 으로 명시** — LLM이 망상으로 채우는 거 방지 + "위에 없는 건 만들지 말 것" 규칙으로 출처 강제

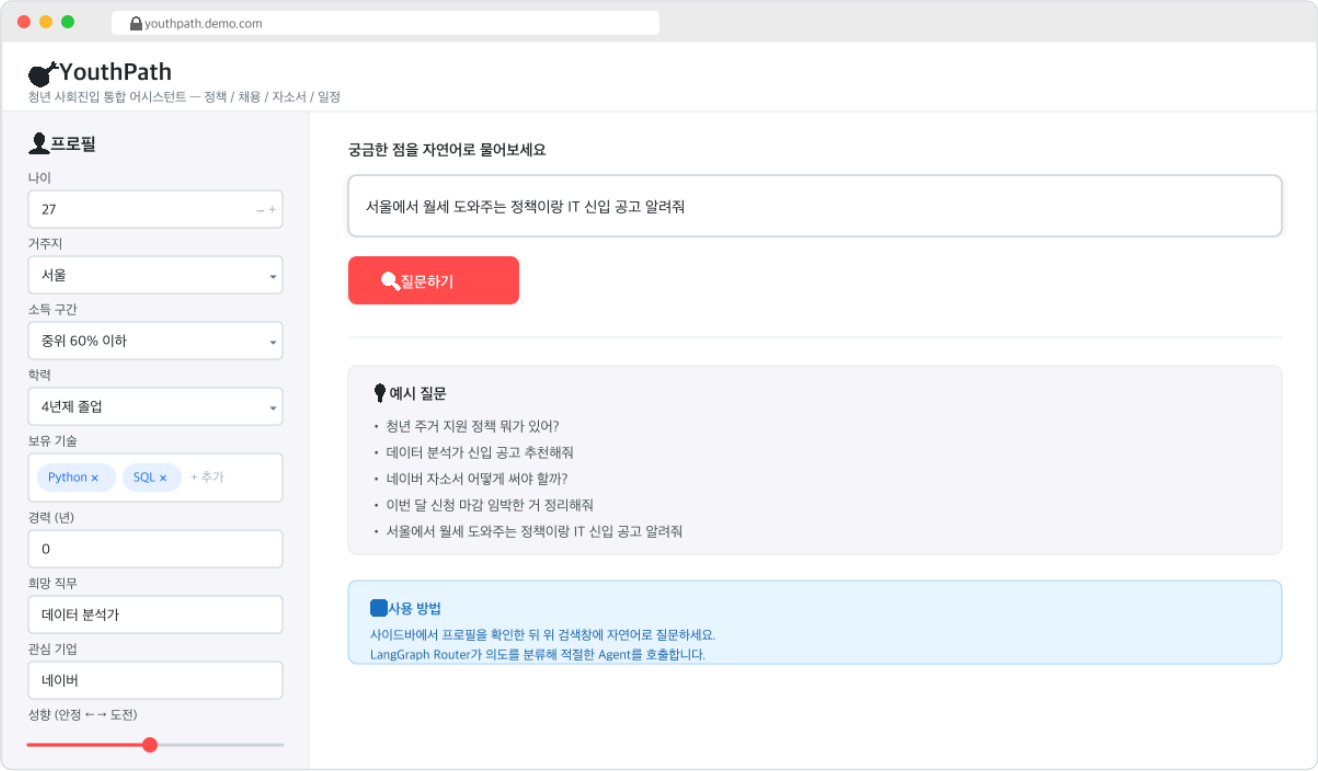
LLM 호출이 2번뿐인 이유

- 비용 절감 (Agent마다 LLM 부르면 호출 폭증)
- 응답 속도 (LLM 호출이 가장 느린 부분)
- 검증 가능성 (Agent 내부는 결정론적이라 테스트 가능)
- 객관성 (LLM은 "이해/표현" 역할, "판단"은 코드)

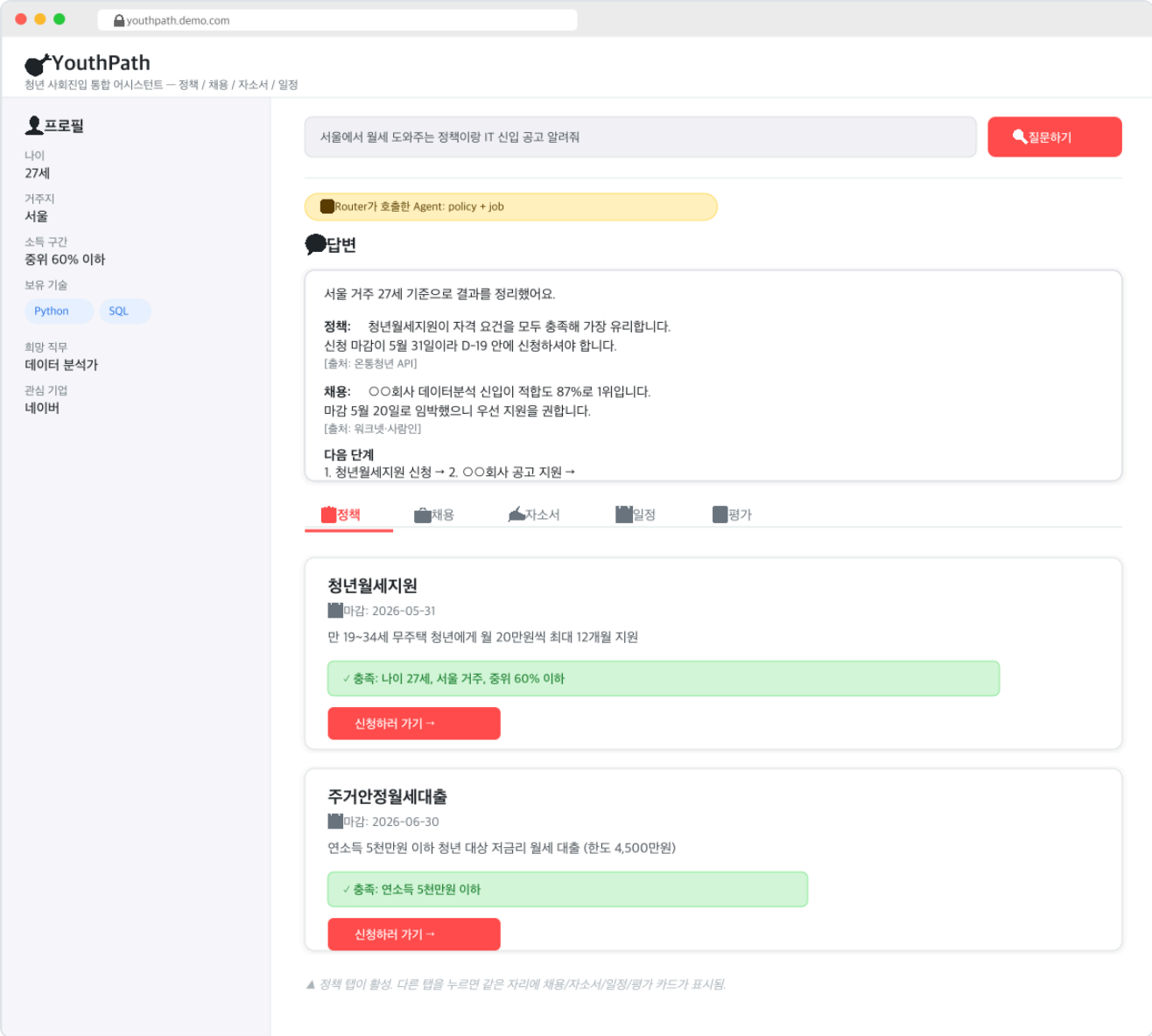
---

11. 화면 배치 (Input / Output)

📥 INPUT 화면



📤 OUTPUT 화면



💼 채용 탭 (탭만 바뀌면 카드도 교체)

```
┌──────────────────────────────────┐
│ ○○회사 — 데이터분석 신입          │
│ ┌──────┬──────┬──────┐          │
│ │ 적합도 │ 위치  │ 마감  │        │ 채용
│ │ 87%  │ 서울 강남 │ 05-20 │      │ 카드 1
│ └──────┴──────┴──────┘          │
│ 요구 스킬: Python, SQL, Tableau   │
│ [ 공고 보기 → ]                   │
└──────────────────────────────────┘

┌──────────────────────────────────┐
│ △△기업 — 주니어 데이터 분석가      │
│   적합도 78% · 서울 판교 · 마감 05-25 │
│ [ 공고 보기 → ]                   │
└──────────────────────────────────┘
```

✍️ 자소서 탭

```
┌──────────────────────────────────────┐
│ 📌 네이버 자소서 가이드                  │
│                                      │
│ 강조할 키워드:                           │
│   검색, AI, 추천 시스템, 광고, 글로벌     │
│                                      │
│ 매칭 포인트:                             │  자소서
│   • Python ↔ AI (적합도 82%)             │  카드
│   • SQL ↔ 추천 시스템 (71%)              │
│                                      │
│ 보강 제안: 대규모 트래픽, 광고 도메인     │
│ Story Angle: 기술적 깊이, 문제 해결       │
└──────────────────────────────────────┘
```

📅 일정 탭 (D-day 짧은 순)

```
┌──────────────────────────────────────┐
│ ○○회사 데이터분석 신입 마감              │
│ ┌──────────┬────────────────┐          │
│ │  D-8     │ 2026-05-20     │          │  일정
│ └──────────┴────────────────┘          │  카드 1
│ [ 바로가기 → ]                          │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ 청년월세지원 마감                        │
│   D-19 · 2026-05-31                  │
└──────────────────────────────────────┘
```

📊 평가 탭

```
📊 시스템 평가 결과 (B1 vs B2 vs B3)

┌──────────────────┬──────┬──────┬──────┐
│ Metric           │ B1   │ B2   │ B3   │
├──────────────────┼──────┼──────┼──────┤
│ Faithfulness     │ 0.42 │ 0.68 │ 0.81 │
│ Context Recall   │ 0.31 │ 0.59 │ 0.74 │
│ Answer Relevance │ 0.55 │ 0.71 │ 0.83 │
└──────────────────┴──────┴──────┴──────┘

▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ B3 (YouthPath)
▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓         B2 (LLM+RAG)
▓▓▓▓▓▓▓▓▓▓▓▓▓▓               B1 (LLM only)
```

핵심 배치 요약

- **왼쪽 사이드바** = 프로필 (한 번 입력하면 세션 내내 유지)

- **상단 메인** = 질문창 + 질문하기 버튼
- **중단** = 호출된 Agent 표시 + 💬 LUXIA 자연어 답변
- **하단** = 5탭 (정책 / 채용 / 자소서 / 일정 / 평가), 탭마다 raw JSON이 카드로 시각화